

Exact Boolean Decomposition Artifacts under Conditioning: Algebraic Contracts and a Reproducibility Audit

Brian Theory

Research draft prepared for author review

26 September 2026 — Draft 3

Abstract

A compact Boolean decomposition is not automatically a useful persistent query representation. We study exact conditioning and model counting with four familiar families: disjoint XOR factors, binary-field matrix factors, cofactor prototypes up to complement, and coordinate-aligned products. Explicit scope and arithmetic contracts yield direct updates without constructing the global residual truth table. Rank-profile identities, factor-only recompression and fixed-cut prototype bounds clarify what can be retained and counted. Counterexamples distinguish a one-query histogram from persistent state and a retained factor list from a freshly decomposed one; a black-box verification bound states the source-access limitation. An executable audit of a supplied implementation reproduces six contract boundaries. A separate scalar checker and reference implementation pass 105,292 exhaustive small-domain conditioning checks, additional matrix and persistence suites, and bounded conversion tests for all four legacy artifact tags. Experiments use 26 outputs of one pinned EPFL control design and 12 synthetic controls. Compact binary encoding changes many selections, but a directly timed 128-query build–save–reload experiment shows no advantage over packed counting in this reference implementation. These results support a bounded methods and reproducibility report, not new decomposition primitives, an optimal encoding, or competitive industrial performance.

1 Question, contribution, and scope

Suppose an exact decomposition of a Boolean function has been found once. Later queries fix input variables and ask how many completions make the function true. Must the computation return to the full truth table after every query? For several standard decompositions the answer is no. The harder questions are what the retained representation must contain, how its correctness is checked, and whether its total cost is worthwhile.

The motivating source is a research portfolio around Correspondence Matrices (CMs)[11]. Its structural-computation folder is explicitly a research direction, not a completed submission. The present report separates a small operational problem from that portfolio’s foundational operator calculus and its pair compiler. It does not propose a new meaning for Boolean rank, recast known matrix identities as new CM theorems, or reuse the pair compiler’s historical performance conclusions.

The contribution is threefold. First, the report states a common contract for exact conditioning and counting in a small heterogeneous portfolio. Second, it gives reproducible tests distinguishing reconstruction integrity from stronger claims about rank, size, immutability, and source equivalence. Third, it preserves an exploratory cost model and repairs it with a directly timed reference persistence replay. The closure facts and elementary counting identities are included with proofs for completeness; historical priority is not claimed for them.

Publication posture. This is a third technical-report draft, prepared for author checking and circulation. The new profile, quotient, and field-gap statements strengthen its explanatory content without asserting historical priority. The computational evidence is deliberately narrow, and the literature does not support presenting “compile once, query repeatedly” or “certified Boolean compilation” as new ideas. Whether the integrated report warrants a research submission remains a separate question from whether its statements and software are useful.

2 Relation to established methods

Knowledge compilation evaluates representations by their succinctness and supported queries and transformations[4]. Conditioning and counting therefore supply an established lens, not a new paradigm. ROBDDs are a central exact symbolic comparator[2]; our implementation of that comparator is explicitly a small fixed-order Python reference, not CUDD.

Disjoint-support decomposition already studies factorizations into functions on separate variable sets[8]. Modern Ashenurst–Curtis decomposition uses cofactor multiplicity and variable partitions for LUT mapping[10]. Its hardware objective is not identical to our scalar-count workload, but it is essential prior art for claims about discovering useful Boolean structure. A recent extension exploits controllability don’t-cares[14]; this extra completion freedom is absent from our fully specified truth functions, and must not be imported silently into exact counting.

Tensor trains have already been analyzed as Boolean knowledge-compilation representations, with conditioning and counting operations[9]. Tensor-network weighted counting also predates that comparison[13]. A factorized tensor with ordinary arithmetic and a factorization over $\mathbb{F}_2$ are not interchangeable objects. Binary matrices have several distinct rank notions[5]. Our matrix multiplication is over $\mathbb{F}_2$, while model counts are ordinary nonnegative integers.

Certified knowledge compilation now includes proof-producing compilation and formally verified checking and counting[3]. A Python reconstruction check does not match that assurance level. Finally, algebraic model counting provides a broader semiring setting[7]; weighted variants are not an empirical contribution of this draft.

3 Source recovery and the operational boundary

The supplied archive and its nested archives were inventoried with SHA-256 hashes and CRC checks. This inventories 3,668 file entries, including nested archive files, with 1,228 distinct content hashes. It does not imply that every line of every neighboring manuscript or log received a substantive audit. The reading map identifies the primary structural notes, the relevant foundation theorem, and the recovered implementation.

The principal code examined is `cmbench/recognition/gf2_decomposition.py` in the recovered `paper_b_current_01/source` snapshot. Its four artifact tags are `xor_components`, `gf2_rank`, `cofactor_blocks`, and `kronecker`. The file advertises $2 \leq n \leq 10$ and a default limit of 64 candidate partitions. The partition family combines an interaction-component proposal, singleton partitions, and further balanced candidates; it is not a global search over every possible decomposition. A minimum returned by the selector means a minimum among generated candidates under its stated score.

The source’s rank cost $r(R + C)$ is an ideal payload count. It is not measured JSON size or resident memory. Source hashes and reconstruction checks are useful integrity mechanisms, but the scope of their assurance must be stated explicitly.

The reference implementation in this report is new code, not an unannounced patch of the historical compiler. Its product artifact uses disjoint table factors. A coordinate-aligned matrix Kronecker product can be represented this way by grouping each factor’s row and column variables. The new discovery procedure tests simple rank-one product splits; it does not reproduce every historical Kronecker search rule. Comparisons concern semantic families, not identical discovery coverage.

4 Definitions and exact artifact semantics

Let $V = (v_0, \dots, v_{n-1})$ be an ordered list of distinct declared variables and $f : \{0, 1\}^V \rightarrow \{0, 1\}$. The first listed variable is the most significant coordinate of an assignment index. The stored truth integer has bit a equal to $f(a)$. Declared variables are not silently removed when they happen to be irrelevant to the function: they still contribute assignments to a model count.

A partial assignment $\rho : T \rightarrow \{0, 1\}$, $T \subseteq V$, gives a residual function $f|_\rho$ on $V \setminus T$ in inherited order. Define

$$N(f, \rho) = |\{a \in \{0, 1\}^{V \setminus T} : f(a, \rho) = 1\}|. \quad (1)$$

The empty variable set has one assignment. Thus a scalar true function has count one, not zero. Fully assigning a function and querying an unassigned constant are required boundary cases.

Definition 4.1 (Exact persistent artifact). An artifact consists of a declared ordered variable universe, a representation tag, and a finite payload with an evaluation map. It is exact for f when its scalar evaluation equals f on every declared assignment. A conditioning operation is correct when its output has the residual variable universe and evaluates to $f|_\rho$. A counting operation returns the integer in (1).

We use four families. In each, the factor scopes or the two matrix scopes form a disjoint cover of the declared universe.

Disjoint XOR. The payload is a bit c and local truth tables g_i on disjoint blocks V_i , with

$$f = c \uparrow g_1 \updownarrow \dots \updownarrow g_k. \quad (2)$$

Empty blocks are permitted after conditioning. The empty product convention used below also covers scalar artifacts.

Binary-field matrix factors. A cut $X \mid Y$ yields a matrix $M_{x,y} = f(x, y)$ with $R = 2^{|X|}$ rows and $C = 2^{|Y|}$ columns. The payload stores

$$M = UW, \quad U \in \mathbb{F}_2^{R \times r}, \quad W \in \mathbb{F}_2^{r \times C}. \quad (3)$$

Here r is an inner dimension. It equals the matrix rank only when minimality has been established. Rank zero is allowed. The standard relationship between $\text{GF}(2)$ rank and XOR sums of separated products is already stated in the neighboring foundations manuscript[12]; it is background here.

Cofactor prototypes. For each row index x , store a prototype index $j(x)$ and flip bit $e(x)$, so

$$M_{x,y} = p_{j(x)}(y) \updownarrow e(x). \quad (4)$$

Prototypes can be canonicalized up to complementation. Storing a prototype table together with references is different from storing the general functional encoding produced by an ACD optimizer.

Disjoint products. For tables g_i on disjoint blocks, store $f = \prod_i g_i$. Since factors are Boolean, multiplication is conjunction. A matrix identity $M = A \otimes B$ has this form after the factor variables are grouped consistently. Arbitrary row permutations and arbitrary row subsets need not preserve the displayed Kronecker shape.

5 Conditioning without a full residual table

Theorem 5.1 (Family-wise conditioning). *Each family above admits an exact conditioning operation which uses only its factors and the assignment ρ . It need not construct the $2^{n-|T|}$ entries of the complete residual truth table. Scalar and one-variable residuals are included.*

Proof. For disjoint XOR and product artifacts, replace each local factor g_i by its restriction to $\rho|_{V_i}$. The defining equality is preserved under substitution. Keep the residual local scope even when the restricted table is constant, or account explicitly for any discarded free variables.

For a rank artifact, let I and J be the row and column assignments compatible with the restrictions on X and Y . The residual matrix is

$$M[I, J] = U[I, :]W[:, J]. \quad (5)$$

The equality follows entry by entry from the original matrix product. Only coefficient rows and basis-column selections are needed.

For prototypes, retain the references for rows in I and restrict every referenced prototype to columns in J . Equal or complementary restricted prototypes may be merged with the flip bits adjusted. This changes the dictionary, not the represented row.

For $A \otimes B$, a Boolean coordinate restriction induces Cartesian selections in the grouped row and column coordinates of each factor. Consequently it selects submatrices of A and B and retains their product. This assertion is made only with the stated coordinate alignment. The disjoint-product representation gives the same argument directly. $\square$

Proposition 5.2 (Basic monotonicity). *Under the explicit updates in the theorem, the stored number of XOR or product factors can be kept constant, GF(2) matrix rank cannot increase, and the number of used prototype classes up to complement cannot increase. If rank factors retain their old inner dimension, their ideal matrix payload $r(R' + C')$ cannot increase.*

Proof. The factor-list update changes tables rather than adding factors. Matrix restriction multiplies by row and column selection matrices, so its rank does not increase. Every new prototype class is the image of an old class under the same column restriction, possibly with classes merged. Finally $R' \leq R$ and $C' \leq C$. $\square$

This proposition says nothing about the size of a JSON audit history, allocator overhead, or the number of factors after a fresh decomposition search. Those are different quantities. Nor does it promise that a residual rank artifact remains the smallest representation in the portfolio.

Example 5.3 (A count-preserving local update). Let $f(x_1, x_2, x_3, x_4) = (x_1x_2) \upharpoonright (x_3x_4)$. Each factor has one satisfying assignment among four, so f has six satisfying assignments among sixteen. Under $x_1 = 0$, the first factor is the constant zero on the still-free variable x_2 . Its zero count is two. The residual function therefore has two satisfying assignments, not one. Dropping x_2 from the declared universe without a multiplicity correction would be a counting error.

6 Direct exact counts

Theorem 6.1 (XOR, product, and prototype counts). *Suppose the factors have already been conditioned. Write z_i, o_i for the numbers of zero and one assignments to the i th factor, and $t_i = z_i + o_i$. Then*

$$N \left(c \updownarrow_i g_i, \emptyset \right) = \frac{\prod_i t_i - (-1)^c \prod_i (z_i - o_i)}{2}, \quad (6)$$

$$N \left(\prod_i g_i, \emptyset \right) = \prod_i o_i. \quad (7)$$

For a prototype artifact with C' residual columns, count

$$\sum_{x \in I} \begin{cases} \text{wt}(p_{j(x)}|_J), & e(x) = 0, \\ C' - \text{wt}(p_{j(x)}|_J), & e(x) = 1. \end{cases} \quad (8)$$

Proof. For a Boolean bit b , $(-1)^b$ is 1 on zero and -1 on one. Disjoint factor scopes make the sum of the sign of (2) factor into $(-1)^c \prod_i (z_i - o_i)$. This sign sum equals the number of zero assignments minus the number of one assignments; the total is $\prod_i t_i$. Solving the two equations gives (6). A product is one exactly when all factors are one, giving (7). A complemented prototype has $C' - \text{wt}(p)$ ones, giving (8). $\square$

Rank factors require parity-aware counting. It is invalid to multiply the ordinary integer sums of U and W : the matrix product in (3) is over $\mathbb{F}_2$. Our reference algorithm groups equal coefficient rows of $U[I, :]$. For each distinct coefficient vector, it XORs the selected basis rows, takes a packed popcount, and multiplies by that coefficient row's multiplicity. It can hold one packed residual row at a time instead of the whole $R' \times C'$ matrix. This is an exact algorithm but not a theorem that low rank makes every query fast. Section 7 gives a profile identity and explicit alternatives, including a factor-only minimal-rank repair.

Cost and output contracts. For explicit local tables, restriction can scan their entries rather than all global assignments. Rank slicing touches at most the coefficient and basis tables, and counting can process distinct coefficient rows. A scalar count and an explicit residual truth vector are different outputs: writing the latter already requires $2^{n-|T|}$ output bits. Avoiding this output does not itself establish an asymptotic or measured advantage over another scalar-count representation. In particular, row streaming can still evaluate every residual entry; the avoided object is a simultaneously stored global table, not necessarily all of its arithmetic work.

Decision queries. Once the count is exact, satisfiability is the test $N > 0$ and tautology is $N = 2^{|V \setminus T|}$. No separate SAT or validity experiment is claimed.

7 Rank profiles, exact counting, and factor-only repair

The contrast between parity arithmetic and integer counting can be stated explicitly. In this section write $M = UW$ over $\mathbb{F}_2$, with inner dimension r . Let $u_x \in \mathbb{F}_2^r$ be a coefficient row of U and $w_y \in \mathbb{F}_2^r$ a column profile of W . For a current restriction, define integer histograms

$$\alpha_a = |\{x \in I : u_x = a\}|, \quad \beta_b = |\{y \in J : w_y = b\}|, \quad a, b \in \mathbb{F}_2^r. \quad (9)$$

The assignment-to-profile association is retained in the artifact; these histograms are computed for a specified query.

Theorem 7.1 (Profile count identity). *With $R' = |I|$ and $C' = |J|$,*

$$N(f, \rho) = \frac{R'C' - \sum_{a,b \in \mathbb{F}_2^r} \alpha_a \beta_b (-1)^{a \cdot b}}{2}. \quad (10)$$

A dense Walsh–Hadamard transform of β evaluates the sign sum using $O(r2^r)$ integer additions/subtractions and $O(2^r)$ final arithmetic operations, after histogram construction. The transform has $O(2^r)$ integer workspace. A row-streaming alternative is polynomial in the explicit factor-table size and needs no global matrix.

Proof. For an entry $M_{xy} = u_x \cdot w_y$ in $\mathbb{F}_2$, its zero/one indicator is

$$\frac{1 - (-1)^{u_x \cdot w_y}}{2}.$$

Sum over the Cartesian product $I \times J$ and group pairs with the same profiles. This gives (10). If $H_{ab} = (-1)^{a \cdot b}$, the sign sum is $\alpha^T H \beta$. The standard butterfly recursion evaluates $H \beta$ in $r2^r$ additions/subtractions. Every transform entry has magnitude at most C' ; the final accumulated count is bounded by $R'C'$.

Alternatively form each represented row by XORing at most r listed basis rows, popcount it, and discard it. Processing every row costs at most $O(rR'C')$ elementary bit operations plus indexing, and uses one additional C' -bit row. Equal coefficient rows may be grouped and processed once. Both R' and C' are explicit factor dimensions, so this is polynomial in the listed input size. The dense transform’s exponential dependence on r is not a hardness result for the representation language. $\square$

The transform is standard[1]; the contribution here is the explicit separation of the two arithmetic roles and its tested use in this artifact contract. Reading a dense pair of factors takes $O(r(R' + C'))$ Boolean entries. Integer arithmetic is not literally unit-cost for unbounded dimensions: counts require up to $\lceil \log_2(R'C' + 1) \rceil$ bits. Nor is the transform always a space improvement; its 2^r workspace can exceed the matrix size. The reference transform is therefore capped and is a verification/alternative algorithm, not the default measured query path.

Example 7.2 (Cancellation rather than integer multiplication). Take $U = [1 \ 1]$ and $W = [1 \ 1]^T$. The represented one-entry matrix is zero over $\mathbb{F}_2$, although the ordinary integer product is two. Thus the model count is zero. Rank recompression should return inner dimension zero, not the stored width two.

Proposition 7.3 (Minimal rank from factors alone). *Given explicit $U \in \mathbb{F}_2^{R' \times r}$ and $W \in \mathbb{F}_2^{r \times C'}$, a minimal factorization of UW can be obtained without forming its $R' \times C'$ entries, using $O((R' + C')r^2)$ field operations with ordinary elimination. A supplied factorization of inner dimension t is minimal exactly when its first factor has full column rank t and its second has full row rank t .*

Proof. Factor $U = AB$ with A full column rank $p \leq r$ and B having p rows. Form $D = BW$, which has at most r rows, and factor it minimally as $D = EF$ with inner dimension t . Then $UW = (AE)F$. Since multiplication by A is injective, AE has column rank t , and F has row rank t . Consequently the product has rank t , so the factorization is minimal. The intermediate tables have factor-sized dimensions, and elimination and multiplication have the stated conservative operation bound. Conversely, a rank deficiency in either factor forces product rank below the common inner dimension; full column and row rank give equality. $\square$

The proof is standard linear algebra, not a new matrix-rank principle. Operationally it repairs a real ambiguity: conditioning preserves a valid factorization but can leave a nonminimal stored width. The independent scalar checker and the separate factor-rank check establish different properties; neither substitutes for the other.

8 Fixed-cut relations between rank and prototype artifacts

Let M have at least one row and column. Write $\mathcal{W} \subseteq \mathbb{F}_2^C$ for its row span, $r = \dim \mathcal{W}$, and

$$\delta = \begin{cases} 1, & \mathbf{1}_C \in \mathcal{W}, \\ 0, & \mathbf{1}_C \notin \mathcal{W}. \end{cases}$$

Let k be the number of equivalence classes among the *occurring rows* of M , where v and $v + \mathbf{1}_C$ are identified. This is the minimum dictionary size when whole row prototypes may be complemented but not otherwise recombined.

Theorem 8.1 (Complement-quotient bounds). *For the fixed cut and explicit-prototype model,*

$$r \leq k + \delta, \quad 1 \leq k \leq \min\{R, 2^{r-\delta}\}. \quad (11)$$

Proof. Choose one occurring row from each class. If $\delta = 0$, two distinct rows of $\mathcal{W}$ cannot differ by $\mathbf{1}_C$, so every occurring row is one of the k chosen rows. They span $\mathcal{W}$, giving $r \leq k$. If $\delta = 1$, every occurring row is a chosen row or that row plus $\mathbf{1}_C$. Hence the chosen rows together with $\mathbf{1}_C$ span $\mathcal{W}$, giving $r \leq k + 1$.

For the upper bound, the kernel of the map $\mathcal{W} \rightarrow \mathbb{F}_2^C / \langle \mathbf{1}_C \rangle$ has dimension δ . Its image has $2^{r-\delta}$ elements. The k occurring classes are a subset of this image and there are at most R occurring rows. At least one row occurs, so $k \geq 1$. $\square$

This refinement is an elementary quotient-space argument. It is proved here and computationally checked, but priority is not asserted. The table model matters: an ACD encoding can compute its selector functions structurally, while this bound counts stored whole prototypes.

Proposition 8.2 (A fixed-cut exponential dictionary gap). *For $x, y \in \mathbb{F}_2^r$, let $M_r(x, y) = x \cdot y$. Then $\text{rank}_{\mathbb{F}_2} M_r = r$ and $k = 2^r$. A rank payload needs $r(2^r + 2^r)$ listed bits, whereas a whole-row prototype dictionary alone needs 2^{2r} bits. At the same cut, the ordinary real rank of the 0/1 matrix is $2^r - 1$, and the sign matrix $((-1)^{x \cdot y})_{x, y}$ has real rank 2^r .*

Proof. The r coordinate rows are independent, and every row is their linear combination, so the GF(2) rank is r . All 2^r rows are distinct. Their $y = 0$ coordinate is zero, so no two are complements; hence $k = 2^r$ and the payload counts follow.

The sign rows are pairwise orthogonal over the reals: the character sum of any nonzero linear functional on $\mathbb{F}_2^r$ is zero. Thus the sign matrix has full real rank. For the 0/1 matrix, remove the zero row and column. For $r \geq 2$, the Gram matrix of the remaining rows is $2^{r-2}(I + J)$: a nonzero linear form is one on 2^{r-1} inputs, and two distinct nonzero forms are simultaneously one on 2^{r-2} inputs. This Gram matrix is positive definite. Thus the remaining $(2^r - 1)$ rows are independent. The $r = 0, 1$ cases are immediate. $\square$

Two qualifications are essential. First, the statement fixes the $x \mid y$ cut and an explicit dictionary model; it does not separate optimally partitioned portfolios. In fact $x \cdot y$ is a disjoint XOR of the r pairwise products $x_i y_i$. Second, the field gap does not refute tensor-train results[9]; it explains why an ordinary-arithmetic tensor bond rank and a GF(2) matrix rank cannot be substituted for each other. A different variable order also changes the relevant tensor cut.

9 What must persist, and what exact checking can establish

The preceding formulas describe a specified query. They do not by themselves identify a sufficient state for all future named-variable queries.

Proposition 9.1 (A profile histogram is not a persistent coordinate index). *The unconditional histograms of coefficient rows and column profiles in a rank factorization need not determine counts after a coordinate restriction, even for two variables and inner dimension one.*

Proof. Let $f(x, y) = xy$ and $g(x, y) = (1 - x)y$. Using row order $x = 0, 1$ and column order $y = 0, 1$, their factorizations have coefficient columns $(0, 1)^T$ and $(1, 0)^T$, respectively, and the same basis row $(0, 1)$. In each case the two profile histograms put multiplicity one on each of 0 and 1. Both unconditional counts are one. But fixing $x = 0$ gives count zero for f and one for g . The histograms have discarded the row-to-assignment association needed to distinguish the queries. $\square$

The reference rank artifact therefore retains assignment-indexed factors. Histograms may be computed for one selected row/column set, or cached together with the state they describe, but they cannot replace that state merely because they give the right unconditional count.

Proposition 9.2 (Exact all-query states distinguish functions). *Let a deterministic state and query procedure answer $N(f, \rho)$ exactly for every partial assignment, including complete assignments, for each f in a supported class $\mathcal{F}$. Distinct functions in $\mathcal{F}$ must have distinct states. Consequently a fixed b -bit state requires $b \geq \lceil \log_2 |\mathcal{F}| \rceil$.*

Proof. If $f \neq g$, there is a complete assignment a with $f(a) \neq g(a)$. The residual universe is empty, so $N(f, a) = f(a)$ and $N(g, a) = g(a)$. A common state could not answer both queries correctly. The bit bound is the elementary count of at most 2^b fixed-width states. $\square$

For all Boolean functions this gives $b \geq 2^n$. It does not rule out strong compression of structured subclasses, variable-length codes with appropriate size guarantees, or smaller summaries for a restricted query family. No historical novelty is claimed for this information-counting argument.

Proposition 9.3 (Black-box exact verification requires a full zero transcript). *Consider a deterministic zero-error verifier given an artifact for the zero function on n variables and only individual value-query access to an unknown source f . To accept exactly when f equals that artifact for every permitted source, it must query all 2^n assignments on the all-zero source. The conclusion remains true under the promise that the source's truth matrix has $GF(2)$ rank at most one across a fixed cut.*

Proof. Suppose some assignment a is unqueried on the all-zero transcript. Replace the source by the function that is one exactly at a . All queries made by the deterministic verifier receive the same answers, so it follows the same path and accepts incorrectly. Across any fixed cut, the one-spike matrix has the form $e_x e_y^T$ and rank one, while the zero matrix has rank zero. Thus the promised class does not remove the adversary. $\square$

This is a statement about an explicit access model. It is not a lower bound against source formulas with proof certificates, authenticated preprocessing, aggregate queries, trusted summaries, or approximate/randomized verification. In an analogous raw-word read model without global summaries, a word that reveals at most w truth bits gives a lower bound of $\lceil 2^n/w \rceil$ reads. A trusted canonical integer object's metadata is not automatically that raw-word model. In particular, this proposition must not be used to dismiss proof-producing certified knowledge compilation[3], or to charge a redundant exhaustive equivalence proof when a flat wrapper retains its identical source value.

Example 9.4 (Rediscovery can increase the finest XOR factor count). Let $f(x, y, z) = xyz \uparrow x \uparrow y$. The ANF interaction hypergraph is connected by the monomial xyz , so a finest disjoint-XOR decomposition has one nonconstant component. Restricting $z = 0$ leaves $x \uparrow y$, with two singleton components. The retained update can still store one two-variable factor; a fresh decomposition can split it into two. Thus monotonicity of a *stored* factor list does not imply monotonicity of the number of freshly discovered components. The assertion follows directly from uniqueness of ANF: monomials spanning a proposed disjoint partition cannot be supplied by either side alone.

Implementation boundary. A condition operation returns a new artifact over the remaining declared variables. A later condition may mention only that remaining universe; assigning an already removed variable is rejected rather than silently interpreted as a fresh query on the original state. The scalar query interface instead evaluates a view of whichever artifact it receives. Both behaviors are covered by separate tests. Exact semantics, mutable API behavior, serialization policy, and a performance model must remain distinct claims.

10 What the archived loader actually certifies

The audit preserves the unmodified source and executes small witness mutations. Table 1 summarizes the outcomes. “Admitted” means that the archived loader accepted the modified document; it does not mean that the original producer generated that document in ordinary use.

Observation	Reproduced consequence
H1: payload cost	A witness with actual producer score 16 accepts declared <code>factor_bits=1</code> after the document digest and ratio are updated.
H2: rank metadata	Appending a zero basis row and increasing the declared rank preserves the truth table and passes admission.
H3: source identity	Changing a basis bit and replacing the bundled source digest with the changed truth’s digest passes admission. No external source was supplied to this loader.
H4: mutability	A frozen dataclass contains a mutable dictionary. Changing a nested basis value after admission changes reconstruction.
H5–H6: residual boundary	Public analyzer calls with $n = 0$ and $n = 1$ are rejected.

Table 1: Contract boundaries of the archived implementation, not evidence that ordinary producer outputs were semantically wrong.

The first two observations separate an exact semantic factorization from a trusted optimization score and a minimum-rank assertion. The third is the ordinary distinction between a self-consistent digest and externally rooted source equivalence. The fourth limits claims of immutable persistence. The last two show that a mathematical closure theorem cannot simply be attributed to the existing schema.

The new scalar checker consumes an externally supplied truth integer and evaluates the witness independently on every assignment. It does not import the producer or the direct query evaluator. This reduces shared implementation logic but is not formal verification and does not eliminate common specification mistakes. The final constructor recursively copies sequences into immutable tuples, including direct caller-supplied lists; aliasing tests verify that changing the original lists does not alter the stored witness. JSON loading checks structure as well as schema, but external-source verification is a separate step. Serialization is not a proof certificate, and the decoder is not advertised as a hardened adversarial-input service.

11 Exploratory experiment

11.1 Frozen endpoint and data

The measured task is an integer model count under a partial assignment, not construction of a residual table. The common input is a truth integer. Circuit-to-truth extraction is performed before timing and is common to all methods; it is not claimed free in an end-to-end circuit pipeline.

All 26 primary outputs of one EPFL control benchmark are used[6]. Its exact pinned source SHA-256 is recorded in the supplement and matches the historical corpus metadata. The source was recovered through a text view and its bytes were hash-checked. Scalar recursive and packed cube evaluations independently agree on the extracted functions, including off-set covers and the constant output. The declared variables of an output are its syntactic cone inputs in source order. This is one related design, not 26 independent benchmark families.

Twelve synthetic controls use parity, inner product, products of pairwise XORs, and pseudorandom truth tables at $n = 4, 8, 12$. Their structure is intentional. They test recognition and query behavior, not natural incidence. For each case, 128 partial assignments are generated with a fixed seed; each variable is fixed with probability one half and then assigned a uniformly chosen bit. These are independent views of the original function, not a cumulative restriction chain.

11.2 Search, baselines, and lifecycle cost

The new search examines at most 16 prescribed partitions, plus an ANF-component XOR artifact and a flat fallback. It chooses the smallest compact JSON serialization, with deterministic tie-breaking. This policy was frozen before the first timed run. A different encoding or cost function can select a different winner.

The packed baseline stores the source truth integer and prepares literal masks. A query intersects the applicable masks and uses native integer popcount. The second baseline is a fixed-order reference ROBDD with reduction and memoized exact counting; it is not an optimized CUDD implementation. The attempted CUDD-binding installation and direct wheel download failed in the runtime. DSD, ACD, optimized tensor-train, and industrial KC baselines were not executed. Their absence prevents a comparative state-of-the-art claim.

For each method, five repetitions rotate method order. The timed setup includes construction or discovery, source-equivalence checking when the representation changes, serialization and parse time, and mask initialization where applicable. The artifact arm constructs a conditioned artifact and then counts it; the other baselines count directly. This is the v1 arm; the v2 amendment adds fused scalar counts and a compact-binary selection arm on the same cases. It also replaces redundant exhaustive flat-wrapper checking by direct source equality. No old measurements are overwritten.

With setup S and average warm query cost t , the descriptive lifecycle model is

$$T(Q) = S + Qt. \quad (12)$$

Results are reported at $Q = 1, 10, 100, 1000$. A finite estimated break-even against baseline (S_b, t_b) requires $t < t_b$ when $S > S_b$, and is then approximately $(S - S_b)/(t_b - t)$. It is not legitimate to infer amortization merely from a compact factor payload.

11.3 Results and their limits

All tested methods returned identical counts. The minimum-JSON selector retained the flat representation on every one of the 26 external outputs. Among the synthetic controls it selected four XOR artifacts, two product artifacts, and six flat representations. No artifact pipeline had a

lower estimated summed-setup cost than the packed baseline at $Q = 1000$; the raw per-case data include the other query counts and all repetitions.

Case at $n = 12$	Selected family	JSON bytes	Raw truth bytes
Parity	XOR	191	512
Inner product	XOR	155	512
Product of pairwise XORs	Product	157	512
Pseudorandom control	Flat	1324	512

Table 2: Storage quantities from the frozen reference experiment. Raw truth bytes exclude a container header. JSON and raw payload are deliberately not conflated.

For example, the $n = 8$ parity artifact occupies 147 JSON bytes, compared with 32 raw truth payload bytes: an ideal structural factorization does not automatically make that container smaller. At $n = 12$, three structured controls do use fewer JSON bytes than the raw truth payload (Table 2). This is an encoding-specific observation, not a lower bound for all compressed formats.

The aggregate query-time ratio is not a useful speedup headline: 32 selected artifacts are flat fallbacks, so most ratios compare nearly the same packed query with extra discovery before it. Nonflat artifacts are slower in this Python condition-then-count implementation. No confidence interval over an industrial population is justified, and no broad failure of compressed computation follows from these measurements.

11.4 Revision sensitivity: fused queries and compact binary storage

The v2 protocol amendment was recorded after the first review and before its new timing runs. It retains the cases, seeds, partition budget, and repetition scheme. One arm uses fused scalar counts with the original JSON selector. A second selects by the actual byte count of an explicit SC bitstream, including tag, scope identifiers, dimensions, partition masks, payload, and padding. Neither is an optimal compressed format. The original v1 results remain part of the evidence.

In the minimum-binary arm, the 26 external cases select ten flat, fourteen product, and two rank representations, rather than twenty-six flat representations. Across all 38 cases, the selection is fifteen flat, seventeen product, two rank, two cofactor, and two XOR. Thus the JSON selection result was not evidence that structural decompositions were absent. One external output, `invA`, occupies 13 binary bytes including metadata versus 16 raw truth payload bytes. At $n = 12$, parity, inner product, and product-of-XOR controls occupy 29, 23, and 29 binary bytes respectively, versus 512 raw truth bytes.

No case in either revised arm has lower modeled summed-setup cost than packed masks at $Q = 1000$. Fusing reduces some artifact overhead but does not change that outcome. These remain same-data sensitivity results, not an independent confirmatory experiment. The binary run was interrupted after 37 completed cases by a tool timeout and resumed at the remaining case; completed raw samples were retained without rerunning or selection.

The v1/v2 time model measures construction, verification, serialization/parse work and warm query batches. Source inspection in the second review identified a further limitation: the packed and ROBDD arms parsed their saved documents but queried their preexisting live objects. They therefore did not establish fresh-instance persistence replay. Those measurements are retained as setup-plus-warm-query proxies, not retrospectively described as complete reload experiments. It is not a cold-disk, fresh-process, or multi-machine benchmark. Resident memory, including

Python object overhead and the packed baseline’s literal-mask index, was not measured. A broader performance claim remains blocked by these limits and the missing optimized comparators.

11.5 Direct fresh-instance replay in the third pass

A third protocol amendment repairs the loading gap rather than only rephrasing it. Each trial constructs or discovers a representation, checks source equivalence where the representation changes, serializes it, discards that live representation, constructs a new instance from the serialized payload, checks it against the external source, and answers all 128 recorded queries. A single timer measures the complete interval; phase timers are also recorded. There is no untimed warm query batch inside this path. Five repetitions rotate the three methods, giving 570 complete trials across the 38 cases.

The artifact arm uses the minimum-binary selector and fused counting. The flat fallback builds masks from the loaded truth payload. The packed loader reconstructs its literal-mask index from its serialized truth integer. The ROBDD loader uses the stored nodes, not a fresh compilation from the source truth. Thus the queried state is actually reconstructed in all three arms. Compilation, checking and index initialization are charged where performed; an unchanged packed source does not require a representation-equivalence proof.

Table 3 gives selected per-case medians of the *directly measured* complete intervals. No artifact pipeline has a lower median than packed counting on any of the 38 cases. This does not prove that no query count can amortize, nor does it compare optimized representations fairly enough for a state-of-the-art claim. In particular, every small or flat case still pays bounded discovery even when that discovery is unhelpful.

Case	Artifact (ms)	Packed (ms)	Reference ROBDD (ms)
EPFL invA, $n = 7$	10.501	0.078	0.822
Parity, $n = 12$	376.152	0.284	11.018
Inner product, $n = 12$	377.157	0.281	10.369
Product of XORs, $n = 12$	439.170	0.251	4.334
Pseudorandom, $n = 12$	1852.250	0.265	15.289

Table 3: Direct build–check–save–reload–recheck–query elapsed time for 128 queries, median of five trials. These are reference implementations on one shared runtime, not optimized-backend or cold-disk comparisons. All cases and repetitions, including unhelpful discovery, are retained in the raw record.

The replay is still in-memory and same-process, not a claim of cold-disk or cold-cache persistence. Circuit extraction and query generation precede the timed endpoint. It reuses the same cases and query seeds, so it is a methodological repair and sensitivity analysis rather than new holdout evidence. The compact artifact format and the JSON baseline formats are not claimed jointly optimal. The operation count, identifier and allocation policies are documented in the supplement.

12 Verification coverage and reproducibility

The exhaustive core suite enumerates every Boolean function for $0 \leq n \leq 3$: $2 + 4 + 16 + 256 = 278$ functions. Across the generated representations it verifies 3,988 artifacts and serialization round trips, rejects 3,988 mismatched external truth vectors, and checks 105,292 conditioned artifact/count pairs. All passed. The v2 extension checks the profile identity and complement-quotient bounds on all 65,536 binary 4×4 matrices. It also passes 3,020 sequential/fused conditioning checks,

2,000 factor-only recompression checks, and 151 binary round trips and trailing-data rejections. Exact rational elimination confirms the field-rank examples for $r = 1, \dots, 5$. These finite checks supplement, not replace, the proofs. Larger benchmark cases are individually verified against their source truth integers.

The third-pass suite additionally passes fresh-instance round trips for all 278 small-domain functions, 7,070 three-backend query comparisons, 510 single-spike rank checks, and the explicit histogram/refinement counterexamples. Malformed structures and out-of-scope assignments are tested, but these small tests do not establish resistance to hostile or resource-exhaustion inputs.

The supplement contains the original archive, the unmodified recovered code, source hashes, the new producer, separate scalar checker, dataset parser, baselines, frozen protocol, raw timing samples, and machine-readable results. No historical P14 speedup is promoted into this experiment. Historical logs that were not replayed retain their historical status.

An executable adapter also converts 675 historical artifacts: 240 rank, 269 cofactor, 162 Kronecker, and four XOR artifacts. All pass the separate source checker and 6,750 direct-query comparisons. This is a bounded compatibility check, not an exhaustive replay of all historical experiments. A fresh reader can run the complete correctness replay without third-party dependencies. An eight-command clean temporary-directory execution record is included, separately from the original raw results. The reference implementation caps its supported declared universe at sixteen variables, but the measured synthetic functions stop at twelve. The compact codec further requires variable identifiers in $0, \dots, 15$; renaming variables is a separate interface step, not a theorem about arbitrary identifiers. A final boundary test also finds that dense truth integers at $n = 14, 16$ hit this runtime's 4,300-digit JSON conversion guard, although the SC binary round trips pass. The guard was not disabled; numerical dimension caps are not guarantees for every serialization path. The measured cases stop at $n = 12$. This is not evidence of useful scaling to arbitrary Boolean formulas.

13 Portfolio boundary and next research gate

The foundations manuscript remains responsible for the CM/LM calculus, logical pairing, higher-arity lifting, signed frames, and the standard rank/separability discussion[12]. The pair-compiler paper retains its compiler rules, S/T/H provenance, and P14 evidence. The present report neither duplicates those results as new contributions nor depends on a new CM algebra.

The small operator-containment lattice, sixteen-operator orbits, and structural retrieval ideas are omitted. They do not strengthen this particular conditioning/counting argument. Retrieval would require a separate independently meaningful task and contemporary comparators, not synthetic labels that simply recover their own mutation generator.

The next decision should be made on evidence. The revised draft supplies rank profiles, quotient bounds, a binary format, fused queries, and legacy adapters. The final revision adds persistence counterexamples, a source-access lower bound and actual fresh-instance replay. Remaining needs are broader external corpora, optimized baselines, resident-memory measurements, independent mathematical review, and a demonstrably differentiated research contribution. A standalone technical report is currently defensible. A distinct research-paper novelty claim and competitive systems result are not yet established.

14 Conclusion

A semantic decomposition, a persistent data structure, an optimization score, and a source-equivalence certificate are different objects. Four classical structural families can support exact

conditioning and counting without constructing the global residual table, provided scopes, scalar cases, and algebra are explicit. That statement survives exhaustive small-domain testing in a new implementation. It does not imply that the historical schema already has the required closure or that the method is faster than packed truth-table counting. In the directly timed 128-query replay, no artifact pipeline outperforms the packed baseline; this narrow negative result is retained without generalizing it to all implementations or workloads. The present evidence supports a bounded and reproducible methods/audit report, with the strongest publication claims deliberately withheld.

Process disclosure. This research draft and its executable checks were prepared with AI assistance. Subsequent specialist-role audit reports are internal role-separated analyses, not external peer review or independent human endorsement. All unresolved correctness, priority, and generalization questions remain visible in the accompanying ledgers.

References

- [1] J. Alman and K. Rao. *Faster Walsh-Hadamard and Discrete Fourier Transforms From Matrix Non-Rigidity*. arXiv:2211.06459, version 2, 2023. <https://arxiv.org/abs/2211.06459>.
- [2] R. E. Bryant. Graph-Based Algorithms for Boolean Function Manipulation. *IEEE Transactions on Computers*, C-35(8):677–691, 1986. <https://www.cs.cmu.edu/~bryant/pubdir/ieeetc86.pdf>.
- [3] R. E. Bryant, W. Nawrocki, J. Avigad, and M. J. H. Heule. *Certified Knowledge Compilation with Application to Formally Verified Model Counting*. arXiv:2501.12906, 2025. <https://arxiv.org/abs/2501.12906>.
- [4] A. Darwiche and P. Marquis. A Knowledge Compilation Map. *Journal of Artificial Intelligence Research*, 17:229–264, 2002. <https://arxiv.org/abs/1106.1819>.
- [5] D. DeSantis, E. Skau, D. P. Truong, and B. Alexandrov. *Factorization of Binary Matrices: Rank Relations, Uniqueness and Model Selection of Boolean Decomposition*. arXiv:2012.10496, version 2, 2021. <https://arxiv.org/abs/2012.10496>.
- [6] Integrated Systems Laboratory, EPFL. *EPFL Combinational Benchmark Suite*. Pinned commit 0060e156826e733d69bf5b3322d1bdd0d03a1f9a; accessed 26 September 2026. MIT license. <https://github.com/lsils/benchmarks>.
- [7] A. Kimmig, G. Van den Broeck, and L. De Raedt. *Algebraic Model Counting*. arXiv:1211.4475, 2012. <https://arxiv.org/abs/1211.4475>.
- [8] A. Mishchenko. *An Approach to Disjoint-Support Decomposition of Logic Functions*. Author-hosted technical report, Portland State University, 2001. https://people.eecs.berkeley.edu/~alanmi/publications/2001/tech01_dsd.pdf.
- [9] R. Onaka, K. Nakamura, M. Nishino, and N. Yasuda. Tensor Decomposition Meets Knowledge Compilation: A Study Comparing Tensor Trains with OBDDs. *Proceedings of AAAI*, 39(14):15109–15117, 2025. DOI: 10.1609/aaai.v39i14.33657.
- [10] A. Tempia Calvino, A. Mishchenko, G. De Micheli, and R. Brayton. *Practical Boolean Decomposition for Delay-driven LUT Mapping*. arXiv:2406.06241, 2024. <https://arxiv.org/abs/2406.06241>.

- [11] B. Theory. *CM Structural Computation and Decomposition: Research Portfolio*. User-supplied archive, `PapersforPublication(2).zip`, 2026. Original bytes and SHA-256 inventory are preserved in this report’s supplement.
- [12] B. Theory. *Correspondence and Logical Matrices: A Boolean Operator Calculus*. Unpublished supplied manuscript, 2026. Rank/separability theorem, lines 1018–1044 of the preserved TeX source.
- [13] J. M. Dudek and M. Y. Vardi. *Parallel Weighted Model Counting with Tensor Networks*. MCW-2020; arXiv:2006.15512, v2, 2021. <https://arxiv.org/abs/2006.15512>.
- [14] B. Hien, M. Walter, A. Tempia Calvino, A. Mishchenko, and R. Wille. *Ashenhurst–Curtis Decomposition Using Don’t Cares*. Author-hosted IWLS paper, 2025. https://people.eecs.berkeley.edu/~alanmi/publications/2025/iwls25_dc.pdf.